

Goricon 2026 풀이

Official Solutions

by
Gori

문제	의도한 난이도	출제자
A 호반우의 파라파라 댄스	Easy	김민환 kimmingim
B 나는 호반우가 좋아!	Easy	김민환 kimmingim
C 호반우의 4구	Easy	김민환 kimmingim
D 호반우의 4구 특훈 (easy)	Easy	김민환 kimmingim
E 호반우 온라인	Medium	박주형 we12223
F 호반우 찾기	Medium	박주형 we12223
G skeep 팬클럽 임무	Hard	박주형 we12223
H 호반우의 4구 특훈 (hard)	Hard	김민환 kimmingim
I 격자 속 고리 찾기	Hard	박주형 we12223
J skeep 문자열 2	Toycartoon	박주형 we12223

A. 호반우의 파라파라 댄스

print

출제진 의도 - **Easy**

- ✓ 처음 푼 팀: **coxld**, 0분
- ✓ 출제자: kimmingim

A. 호반우의 파라파라 댄스



✓ KNU yaho! Let 's dance Para Para! N번 출력하시면 됩니다.

B. 나는 호반우가 좋아!

sorting

출제진 의도 - **Easy**

- ✓ 처음 푼 팀: **jkrt**, 1분
- ✓ 출제자: kimmingim

- ✓ **접근 방법:** 문자열 내에 포함된 'H'를 최우선으로 두고, 나머지 문자는 알파벳 순서대로 정렬하는 문제입니다.
- ✓ **풀이 1 (일반 정렬):** 'H'를 제외한 나머지 문자를 모아 $\mathcal{O}(N \log N)$ 만에 정렬한 뒤, 'H'를 기존 개수만큼 먼저 출력하고 나머지 정렬된 문자열을 출력합니다.
- ✓ **풀이 2 (카운팅 정렬):** 알파벳은 총 26자뿐이므로, 길이가 26인 배열을 만들어 각 문자의 등장 횟수를 세면 시간 복잡도 $\mathcal{O}(N)$ 만에 훨씬 빠르게 해결할 수 있습니다.

C. 호반우의 4구

geometry, implementation, case_work

출제진 의도 – **Easy**

- ✓ 처음 푼 팀: **jwpassion1**, 5분
- ✓ 출제자: kimmingim

✓ 풀이 1 (벡터의 내적):

1. **조건 확인:** W 와 R_1 의 좌표(x 또는 y)가 같은 경우, R_1 을 중심으로 $\angle WR_1R_2$ 를 확인합니다. (R_2 와 같은 경우는 반대로 확인)
 2. **일직선 예외 처리 (180°):** 세 공의 x 좌표가 모두 같거나, y 좌표가 모두 같으면 기본구의 조건(180 도 미만)을 만족하지 못하므로 제외합니다.
 3. **둔각 판별:** R_1 중심일 때, 두 벡터 $R_1\vec{W}$ 와 $R_1\vec{R}_2$ 가 이루는 각이 둔각이 되려면 **내적값이 음수** (< 0)여야 합니다.
 4. 즉, $(x_W - x_{R_1}) \times (x_{R_2} - x_{R_1}) + (y_W - y_{R_1}) \times (y_{R_2} - y_{R_1}) < 0$ 인지 판별합니다.
- ✓ **시간 복잡도:** 단순 사칙연산만을 수행하므로 $\mathcal{O}(1)$ 입니다.

C. 호반우의 4구

- ✓ **풀이 2 (조건 분기):** 공들의 상대적인 위치(상, 하, 좌, 우)를 직접 비교하여 둔각 여부를 판별하는 직관적인 풀이입니다.
- ✓ **판별 로직:**
 1. 입력으로 주어진 R_1 이 W 와 x 또는 y 좌표가 같은지 확인합니다.
 2. W 에서 R_1 을 향하는 방향을 확인합니다.
 3. 예: $x_W = x_{R_1}$ 이고 R_1 이 W 보다 **위쪽**에 있다면 ($y_W < y_{R_1}$), 둔각을 만들기 위해 나머지 공 R_2 는 R_1 보다 **더 위쪽**에 있어야 합니다 ($y_{R_2} > y_{R_1}$).
 4. 상, 하, 좌, 우 4가지 방향에 대해 모두 이 논리를 적용하고, R_2 가 기준이 되는 경우에 대해서도 동일하게 검사합니다.
- ✓ **일직선 예외 처리 (180°):** 세 공이 나란히 일직선으로 배치되는 것을 막기 위해, R_2 가 W, R_1 과 같은 선상에 있지 않음 ($x_{R_2} \neq x_{R_1}$ 또는 $y_{R_2} \neq y_{R_1}$)을 조건에 추가합니다.

D. 호반우의 4구 특훈 (easy)

bfs

출제진 의도 – Easy

- ✓ 처음 푼 팀: **woohyunjng**, 10분
- ✓ 출제자: kimmingim

D. 호반우의 4구 특훈 (easy)

- ✓ **접근 방법:** 최단 거리(최소 타수)를 구하는 문제이며, 이동 가중치가 모두 1(한 번의 타격)이므로 **BFS(너비 우선 탐색)**를 사용합니다.
- ✓ **핵심 로직 (상태 관리):**
 1. 찬스를 얻기 전과 후의 이동 반경이 다릅니다. 따라서 찬스 획득 여부를 분리해야 합니다.
 2. `visited[x][y][찬스 유무]` 형태의 3차원 배열(크기 $1000 \times 1000 \times 2$)을 만들어 방문 처리를 관리합니다.
 3. 큐(Queue)에는 $(x, y, \text{찬스 유무})$ 정보를 함께 넣고 BFS를 돌립니다.
 4. 찬스가 없는 상태($d=0$)라면 기본 이동($\pm A, \pm B$)만 수행하고, 찬스가 있는 상태($d=1$)라면 기본 이동에 더해 찬스 이동($\pm C$)까지 모두 큐에 추가로 탐색합니다.

✓ 예외 및 주의 처리:

1. 이동하려는 다음 좌표(n_x, n_y)가 당구대 범위($1 \leq x \leq N, 1 \leq y \leq M$)를 벗어나는지 반드시 체크해야 합니다. (벽을 맞고 튕기지 않음)
2. 이동 방향에 주의해야 합니다. 문제 지문에서 A 는 y 축 이동, B 는 x 축 이동이라고 명시했으므로, 배열의 인덱스 방향을 헷갈리지 않게 조심해야 합니다.

- ✓ **시간 복잡도:** 당구대의 최대 넓이는 $N \times M = 10^6$ 이고 상태가 2개이므로 최대 상태 수는 2×10^6 입니다. 각 상태에서 최대 8번 탐색하므로 시간 내에 충분히 통과하는 $\mathcal{O}(NM)$ 입니다.

D. 호반우의 4구 특훈 (easy)

- ✓ **별해:** 한 번의 타격으로 x 축 또는 y 축 중 하나로만 이동합니다. 즉, 2차원 공간을 탐색할 필요 없이 x 축과 y 축 이동을 완전히 분리해서 계산할 수 있습니다.
- ✓ **1. 찬스 미사용 ($S \rightarrow E$)**
 - x 축 거리 차이가 B 의 배수, y 축 거리 차이가 A 의 배수인지(모듈러 연산) 확인하여 나눗셈으로 즉시 타수를 계산합니다.
- ✓ **2. 찬스 사용 ($S \rightarrow D \rightarrow E$)**
 - $S \rightarrow D$ 구간은 위와 동일하게 계산 가능 여부를 판별합니다.
 - $D \rightarrow E$ 구간은 축별로 나눕니다. x 축은 $1 \sim N$ 사이의 경유점을 잡고 B 와 C 의 조합으로, y 축은 $1 \sim M$ 사이의 경유점을 잡고 A 와 C 의 조합으로 도달하는 최소 타수를 각각 구합니다.
- ✓ **결론:** 1차원 순회와 모듈러 연산만으로 $\mathcal{O}(N + M)$ 만에 해결할 수 있습니다.

E. 호반우 온라인

adhoc, constructive

출제진 의도 – **Medium**

- ✓ 처음 푼 팀: **middle_man**, 5분
- ✓ 출제자: we12223



이 문제의 모티브가 된 스즈미야 하루히입니다.

모든 순열을 단순히 이어 붙이면 어떨까요?

- ✓ 총길이는 $N! \times N$ 입니다.
- ✓ $N = 7$ 이면 35 280이므로 제한을 초과합니다.
- ✓ 순열 사이에 겹치는 부분을 활용하여 길이를 줄여 봅시다.

E. 호반우 온라인

순열 $p_1 p_2 \cdots p_N$ 을 하나 선택해 봅시다.

- ✓ 뒤에 p_1 을 붙이면, 마지막 N 개 문자는 $p_2 p_3 \cdots p_N p_1$ 입니다.
- ✓ 직전 순열의 첫 문자를 뒤에 붙이는 과정을 총 $N - 1$ 번 반복합니다.

$$p_1 p_2 \cdots p_N p_1 p_2 \cdots p_{N-1}$$

- ✓ 길이 $2N - 1$ 의 문자열에 서로 다른 순열 N 개가 포함됩니다.
- ✓ 예를 들어, 12312에는 123, 231, 312가 포함됩니다.
- ✓ 이 N 개 중 어느 순열에서 시작해도 같은 N 개의 순열을 얻습니다.

E. 호반우 온라인

아직 포함되지 않은 순열을 골라 같은 작업을 반복합니다.

- ✓ 앞의 방법으로 함께 얻는 N 개의 순열을 하나의 그룹으로 묶습니다.
 - ✓ 모든 순열은 서로 겹치지 않는 그룹들로 나뉩니다.
 - ✓ 그룹은 $\frac{N!}{N} = (N - 1)!$ 개이며, 한 번의 작업으로 한 그룹 전체를 포함시킵니다.
 - ✓ 이미 처리한 그룹을 다시 선택하지 않으므로, 최대 $(N - 1)!$ 번이면 충분합니다.
- 만들어진 문자열들을 이어 붙이면 총길이는

$$|S| \leq (N - 1)!(2N - 1)$$

입니다. $N = 7$ 일 때도 $6! \times 13 = 9\,360 \leq 10\,000$ 이므로 길이 제한을 만족합니다.

F. 호반우 찾기

dynamic_programming

출제진 의도 – **Medium**

- ✓ 처음 푼 팀: **middle_man**, 14분
- ✓ 출제자: we12223

F. 호반우 찾기

$T = \text{hobanwoo}$ 라 하고, 다음과 같이 DP를 정의합니다.

- ✓ $\text{dp}[i][k]$: 앞의 i 개 문자에서 부분 수열을 골라, T 의 k 번째 위치까지 채웠을 때의 최대 점수입니다.
- ✓ 각 위치의 문자는 한 번 이상 선택하며, 여러 번 반복할 수 있습니다.
- ✓ 모든 상태를 $-\infty$ 로 초기화합니다.
- ✓ $S_i = \text{h}$ 이면 $\text{dp}[i][1] = p_h$ 로 새로 시작할 수 있습니다.

현재 문자를 건너뛰거나, 같은 위치에 추가하거나, 다음 위치로 넘어가는 경우를 생각하면 됩니다.

F. 호반우 찾기

- ✓ 현재 문자를 선택하지 않습니다.

$$dp[i][k] = \max(dp[i][k], dp[i-1][k])$$

- ✓ $S_i = T_k$ 이면 같은 위치에 문자를 추가합니다.

$$dp[i][k] = \max(dp[i][k], dp[i-1][k] + p_{S_i})$$

- ✓ $S_i = T_k$ 이고 $k \geq 2$ 이면 이전 위치에서 넘어옵니다.

$$dp[i][k] = \max(dp[i][k], dp[i-1][k-1] + p_{S_i})$$

F. 호반우 찾기

최종 결과는 $dp[N][8]$ 로 판정합니다.

- ✓ $dp[N][8] = -\infty$ 이면 No를 출력합니다.
- ✓ 아니라면 Yes와 $dp[N][8]$ 을 출력합니다.

시간 복잡도는 $\mathcal{O}(N)$ 입니다.

G. skeep 팬클럽 임무

bitmask, tree_dp

출제진 의도 - Hard

- ✓ 처음 푼 팀: **potato167**, 27분
- ✓ 출제자: we12223

쿼리의 조건을 비트별로 나누어 봅시다.

- ✓ $(a_u \& m) \neq (a_v \& m)$ 이려면, m 에서 켜진 비트 중 두 지점의 값이 다른 비트가 하나 이상 있어야 합니다.
- ✓ 각 비트의 값을 0, 1 두 색으로 생각합니다.
- ✓ $\text{ans}[b]$ 에 b 번째 비트에서 서로 다른 색의 두 지점 사이 최대 거리를 저장합니다.
- ✓ $a_u = 0$ 인 지점은 끝점에서 제외하며, 각 $\text{ans}[b]$ 는 0 으로 초기화합니다.

비트 하나를 고정하고 트리 DP를 계산합니다.

- ✓ $dp[u][c]$: u 에서 자신의 서브트리에 있는 선택 가능한 색 c 의 지점까지의 최대 거리입니다.
- ✓ 선택 가능한 정점은 자신의 색을 0, 나머지 상태를 -1 로 초기화합니다.
- ✓ $a_u = 0$ 이면 두 상태를 모두 -1 로 두지만, 경로는 이 지점을 통과할 수 있습니다.

자식 v 와의 간선 길이가 w 이고 $dp[v][c] \geq 0$ 이면,

$$dp[u][c] = \max(dp[u][c], dp[v][c] + w)$$

로 갱신합니다.

G. skeep 팬클럽 임무

u 를 지나는 경로의 최대 거리를 구합니다.

- ✓ 후보는 선택 가능한 자기 자신과 각 자식 서브트리에서 얻습니다.
- ✓ 색마다 거리 상위 두 개와 출처를 저장합니다.
- ✓ 자기 자신은 출처 0, 각 자식은 서로 다른 번호를 사용합니다.
- ✓ 서로 다른 출처의 두 색 후보를 골라 다음과 같이 갱신합니다.

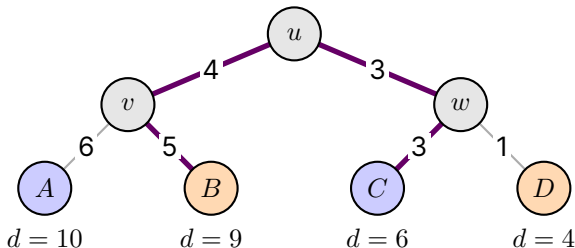
$$\text{ans}[b] = \max(\text{ans}[b], d_0 + d_1)$$

상대 후보와 같은 출처 하나만 피하면 되므로, 색마다 상위 두 개를 저장하여 최대 네 조합만 확인하면 충분합니다.

G. skip 팬클럽 임무



다음 트리를 예시로 봅시다. 간선의 숫자는 길이입니다.



파랑은 색 0, 주황은 색 1, 회색은 선택 불가능한 정점입니다. d 는 u 로부터의 거리입니다. 굵게 표시한 경로의 길이는 $5 + 4 + 3 + 3 = 15$ 입니다.

u 에서 색별로 저장하는 후보는 다음과 같습니다. 각 후보를 (거리, 출처)로 나타냅니다.

색	첫 번째 후보	두 번째 후보
0	$A : (10, v)$	$C : (6, w)$
1	$B : (9, v)$	$D : (4, w)$

- ✓ A, B 는 같은 자식에서 왔으므로 $10 + 9 = 19$ 로 합치면 안 됩니다. 실제 거리는 11이며, 이 경로는 자식 v 에서 이미 확인합니다.
- ✓ 서로 다른 출처인 조합은 A, D 와 C, B 입니다.
- ✓ 각각 $10 + 4 = 14, 6 + 9 = 15$ 이므로 최대 거리는 15입니다.

전처리가 끝나면 각 쿼리는 다음과 같이 처리합니다.

- ✓ m 에서 켜진 비트들의 $\text{ans}[b]$ 중 최댓값을 구합니다.
- ✓ 최댓값이 0이면 가능한 쌍이 없으므로 -1 을 출력합니다.

비트 0부터 30까지 처리하므로, 총 시간 복잡도는 $\mathcal{O}(31(N + Q))$ 입니다.

별개로 **트리의 지름**을 이용할 수도 있습니다. 각 색의 정점 집합에서 지름의 양 끝점을 구한 뒤, 서로 다른 색의 끝점 사이 거리를 비교하면 됩니다.

H. 호반우의 4구 특훈 (hard)

math, number_theory, extended_euclidean, case_work

출제진 의도 – **Hard**

- ✓ 처음 푼 팀: **woohyunjng**, 46분
- ✓ 출제자: kimmingim

H. 호반우의 4구 특훈 (hard)



- ✓ **접근 방법:** 기술 1의 사용 횟수를 x , 기술 2를 y 라 하면, 목표 좌표에 도달하기 위한 2원 1차 연립방정식이 도출됩니다.

$$Ax + Cy = E_x$$

$$Bx + Dy = E_y$$

- ✓ **핵심 1 (크래머 공식):** 행렬식 $\det = AD - BC$ 를 확인합니다.
1. $\det \neq 0$ 인 경우: 해가 **유일**합니다. 크래머 공식을 통해 x, y 를 $\mathcal{O}(1)$ 만에 구합니다.
 2. x, y 가 모두 $\det$ 로 나누어 떨어지고, 음이 아닌 정수 ($x \geq 0, y \geq 0$)라면 그 합인 $x + y$ 가 정답입니다.

- ✓ **핵심 2 (평행과 확장 유클리드):** $\det = 0$ 이면 두 기술의 방향이 평행합니다.
1. 목표점 (E_x, E_y) 이 같은 직선 상에 있는지 확인합니다. (기울기 비교)
 2. 같은 직선 상에 있다면, 식은 $Ax + Cy = E_x$ 하나로 단순화되며, 무수히 많은 해를 가질 수 있는 **선형 디오판토스 방정식**이 됩니다.
 3. 횡수 조건을 만족하는 정수해를 찾기 위해 **확장 유클리드 호제법**(`ext_gcd`)을 사용하여 특수해 (x_0, y_0) 를 $\mathcal{O}(\log N)$ 만에 구합니다.
 4. 단, E_x 가 $\gcd(A, C)$ 의 배수여야 해가 존재합니다.

✓ 핵심 3-1 (일반해와 k 의 범위):

1. $g = \gcd(A, C)$ 라 할 때, 일반해는 다음과 같습니다. (k 는 정수)

$$x = x_0 + k \times \frac{C}{g}, \quad y = y_0 - k \times \frac{A}{g}$$

2. 횃수는 음수가 될 수 없으므로 $x \geq 0, y \geq 0$ 조건을 만족해야 합니다. 이를 통해 k 의 가능한 범위(최솟값 mnk , 최댓값 mxk)를 도출합니다.
3. 만약 $mnk > mxk$ 라면 조건을 만족하는 정수 k 가 없으므로 도달이 불가능한 경우(-1 출력)입니다.

✓ 핵심 3-2 (최소 타수 찾기):

1. 앞서 구한 k 의 범위 내에서, 우리가 최소화해야 할 총 횡수 $x + y$ 를 식별합니다.

$$x + y = (x_0 + y_0) + k \times \left(\frac{C - A}{g} \right)$$

2. 위 식은 변수 k 에 대한 1차 함수 형태입니다.
3. 따라서 기울기에 해당하는 $(C - A)/g$ 의 부호(즉, $C - A$ 의 부호)에 따라 양 끝값 ($k = mnk$ 또는 mxk) 중 하나를 대입했을 때 최솟값을 가집니다.

H. 호반우의 4구 특훈 (hard)

✓ 단계별 시간 복잡도 분석:

1. **크래머 공식 (해가 유일한 경우):** 행렬식을 계산하고 나눗셈을 수행하는 과정은 단순 사칙연산이므로 $\mathcal{O}(1)$ 입니다.
2. **확장 유클리드 호제법 (평행한 경우):** `ext_gcd` 함수의 시간 복잡도는 일반적인 유클리드 호제법과 동일하게 $\mathcal{O}(\log(\min(A, C)))$ 입니다.
3. **최솟값 도출:** k 의 범위를 구하고 식에 대입하는 과정 역시 $\mathcal{O}(1)$ 입니다.

✓ 총 시간 복잡도:

- 각 테스트 케이스당 최대 $\mathcal{O}(\log(\max(A, B, C, D)))$ 의 시간이 소요됩니다.
- 총 테스트 케이스의 수 $T \leq 200,000$ 이므로, 전체 시간 복잡도는 $\mathcal{O}(T \log(\max))$ 입니다.

I. 격자 속 고리 찾기

maximum_flow

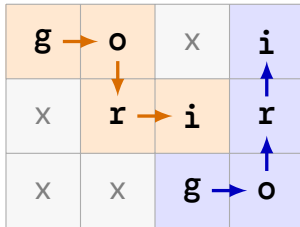
출제진 의도 - Hard

- ✓ 처음 풀 팀: **coxld**, 17분
- ✓ 출제자: we12223

I. 격자 속 고리 찾기



서로 칸을 공유하지 않는 gori를 최대한 많이 찾아야 합니다.

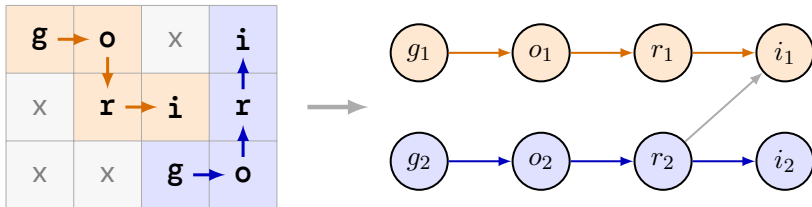


- ✓ 한 경로에서 선택한 네 칸은 다시 쓸 수 없습니다.
- ✓ 그림에서는 두 색의 경로를 골라 2점을 얻습니다.

I. 격자 속 고리 찾기



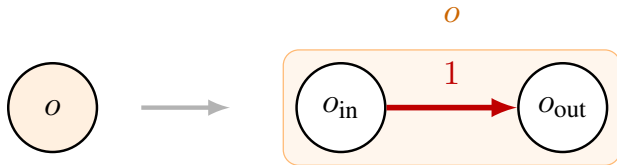
각 칸을 정점으로 보고, 다음 문자로 이동하는 간선을 만듭니다.



- ✓ 인접한 칸을 g, o, r, i 순서로 연결합니다.
- ✓ g에서 i까지의 경로 하나가 gori 하나입니다.

I. 격자 속 고리 찾기

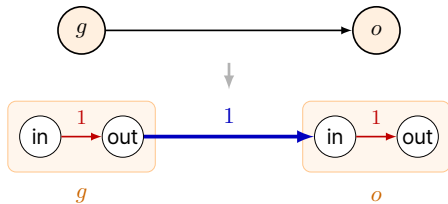
같은 칸을 여러 번 사용하지 못하도록 정점을 분할합니다.



- ✓ 각 칸을 in과 out으로 나누고, 사이의 용량을 1로 둡니다.

I. 격자 속 고리 찾기

기존 간선은 이전 칸의 출구에서 다음 칸의 입구로 연결합니다.

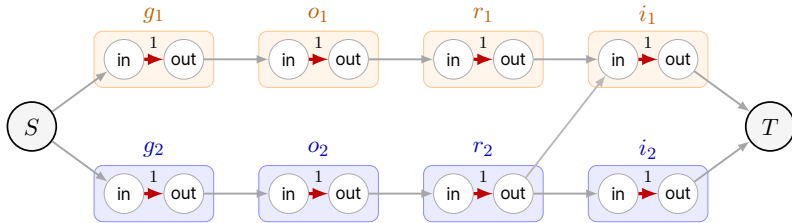


- ✓ g 의 출구에서 인접한 o 의 입구로 연결합니다.
- ✓ o 와 r , r 와 i 도 같은 방법으로 연결합니다.

I. 격자 속 고리 찾기



여기에 시작점 S 와 도착점 T 를 추가합니다.

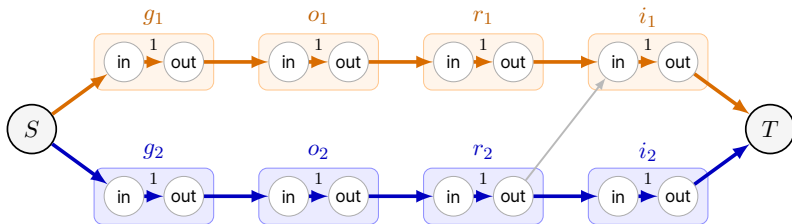


- ✓ S 는 모든 g 의 입구에, 모든 i 의 출구는 T 에 연결합니다.
- ✓ 모든 용량은 1입니다. 역방향 간선은 생략했습니다.

I. 격자 속 고리 찾기



이 그래프의 최대 유량이 정답입니다. 그림에서는 2입니다.



- ✓ 겹치지 않는 gori 하나마다 유량 1을 보냅니다.
- ✓ 반대로 유량 경로들도 칸이 겹치지 않는 gori입니다.

I. 격자 속 고리 찾기



에드몬드-카프 알고리즘으로 최대 유량을 구합니다.
종료했을 때의 유량이 찾을 수 있는 gori의 최대 개수입니다.

종료했을 때의 유량이 찾을 수 있는 gori의 최대 개수입니다. 최대 유량을 F 라고 합시다.

$$F \leq \left\lfloor \frac{NM}{4} \right\rfloor \leq 625$$

한 경로가 네 칸을 쓰므로 위 부등식이 성립합니다.

마지막 실패까지 BFS는 $F + 1$ 번입니다.

$$\mathcal{O}((F + 1)(V + E)) = \mathcal{O}((NM)^2)$$

J. sleep 문자열 2

dynamic_programming

출제진 의도 – [Toycartoon](#)

- ✓ 처음 푼 팀: **potato167**, 127분
- ✓ 출제자: we12223



난이도의 모티브가 된 toycartoon입니다.

원래 문자열을 한 문자로 줄어드는 구간들로 나누어 생각합시다.

- ✓ 길이 L 인 구간이 한 문자로 줄어든다면, 작업 횟수는 항상 $(L - 1)/4$ 입니다.
- ✓ 연속한 부분을 합치는 작업이므로, 마지막에 남는 문자들은 원래 문자열의 서로 겹치지 않는 구간들에 대응합니다.
- ✓ 따라서 각 구간의 축약 가능 여부를 구한 뒤, 작업 횟수의 합을 최대화하면 됩니다.

$S[i, j) = S_i \cdots S_{j-1}$ 이라 합시다. $dp[i]$ 는 앞의 i 개 문자에서 가능한 최대 작업 횟수이며, 초기값은 모두 0입니다.

한 문자를 그대로 남기는 경우,

$$dp[i + 1] = \max(dp[i + 1], dp[i])$$

$S[i, j)$ 를 한 문자로 줄일 수 있는 경우,

$$dp[j] = \max\left(dp[j], dp[i] + \frac{j - i - 1}{4}\right)$$

어떤 구간이 한 문자로 줄어둘 수 있을까요? 길이가 1 보다 큰 구간의 첫 문자를 포함하는 작업들을 봅시다.

- ✓ 첫 문자는 s 여야 하며, 오른쪽의 keep과 합쳐집니다.
- ✓ 이후에도 합쳐져야 한다면, 작업 결과는 다시 s 여야 합니다.

skeepkeep $\rightarrow$ skeep $\rightarrow$ x

각 keep은 해당 구간 안에서 먼저 만들 수 있습니다. 따라서 첫 s 뒤를 keep의 반복으로 만들 수 있는지 판정하면 됩니다.

이를 위해 다음 두 DP를 정의합니다.

- ✓ $k[i][j]$: $S[i, j)$ 를 keep으로 만들 수 있는가?
- ✓ $c[i][j]$: $S[i, j)$ 를 keep의 한 번 이상 반복으로 만들 수 있는가?

$j - i > 1$ 인 구간 $S[i, j)$ 의 축약 가능 조건은 다음과 같습니다.

$$S_i = s \quad \text{이고} \quad c[i+1][j] = 1.$$

두 DP는 모두 0으로 초기화합니다. keep의 각 문자도 다른 구간을 줄인 결과일 수 있습니다.

시작점 i 를 고정하고 $W = \text{keep}$ 이라 둡시다.

$D[q][j]$ 는 $S[i, j)$ 를 W 의 앞 q 개 문자로 만들 수 있으면 1, 그렇지 않으면 0입니다.

처음에는 아무 문자도 만들지 않은 상태입니다.

$$D[0][i] = 1$$

나머지는 모두 0으로 초기화합니다.

$q = 0, 1, 2, 3$ 순서로 다음 문자를 만드는 경우를 계산합니다.

$q = 0, 1, 2, 3$ 에 대해 $j < N, D[q][j] = 1$ 인 상태를 봅시다.

$S_j = W_q$ 이면 문자 하나를 그대로 사용합니다.

$$D[q+1][j+1] = 1$$

$S_j = s$ 이고 $c[j+1][t] = 1$ 이면 구간을 한 문자로 줄입니다.

$$D[q+1][t] = 1$$

네 문자를 모두 처리한 결과를 저장합니다.

$$k[i][t] = D[4][t]$$

c에는 keep 하나만 만드는 경우를 먼저 반영합니다.

$$c[i][t] = k[i][t]$$

$i < j < t$ 에서 $k[i][j] = 1$ 이고 $c[j][t] = 1$ 이면, 두 구간을 이어 붙입니다.

$$c[i][t] = 1$$

시작점 i 를 오른쪽부터 처리하면, 필요한 오른쪽 구간의 DP 값은 이미 계산되어 있습니다.

이 점화식을 그대로 계산하면 $\mathcal{O}(N^3)$ 이 걸립니다.

- ✓ 시작점 i 마다 현재 위치 j 와 다음 끝점 t 를 순회하기 때문입니다.
- ✓ $N = 2000$ 이므로, 모든 끝점을 하나씩 갱신하는 방식은 너무 느립니다.

어떻게 하면 최적화할 수 있을까요?

한 전이에서 수행하는 일은 다른 DP 행에 표시된 끝점들을 현재 행에 모두 추가하는 것입니다. 이를 비트셋의 OR 연산으로 처리합니다.

각 DP 행을 길이 $N + 1$ 인 비트셋으로 나타냅니다. $j < N, D[q][j] = 1, S_j = s$ 인 경우,

$$D[q + 1] = D[q + 1] \vee c[j + 1].$$

마찬가지로 $c[i] = k[i]$ 로 두고, $k[i][j] = 1$ 인 각 j 에 대해

$$c[i] = c[i] \vee c[j]$$

를 계산합니다. 두 행 사이의 $\vee$ 는 비트별 OR 연산입니다. 모든 끝점에 대한 갱신을 한꺼번에 수행할 수 있습니다. 이후 dp를 계산하면 최종 답은 $dp[N]$ 입니다.

전체 시간 복잡도는 $\mathcal{O}(N^3/64)$ 입니다.